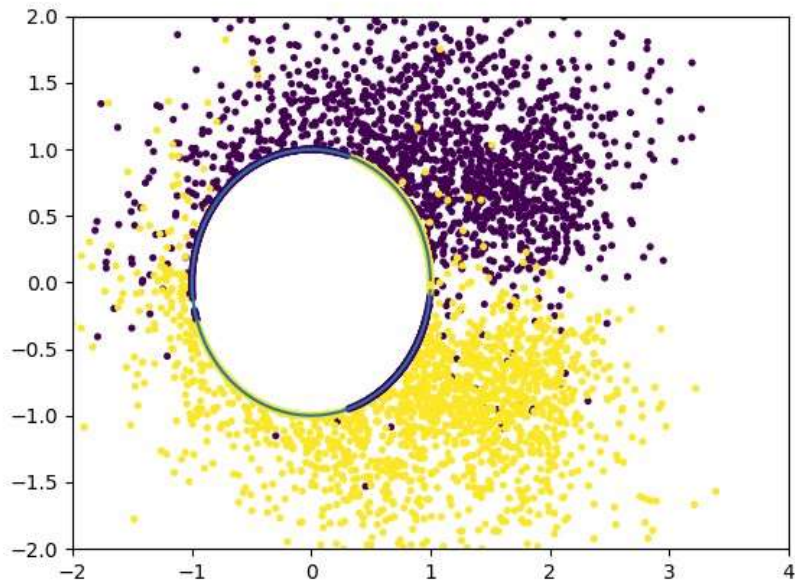
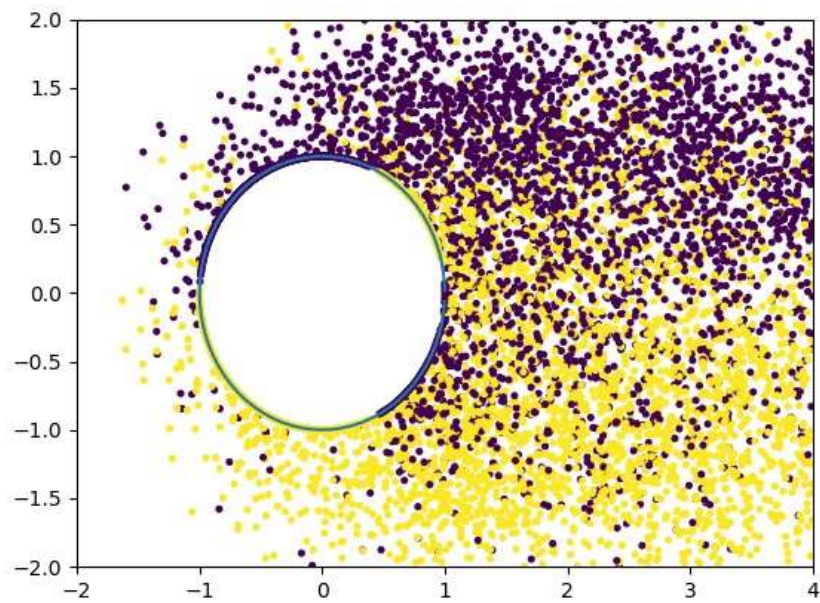


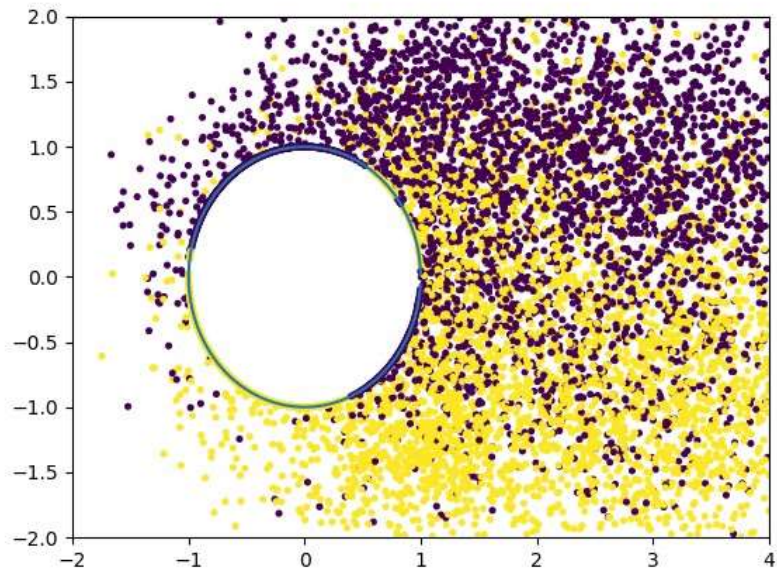
- 1) Plots at each sec
a) Just after start



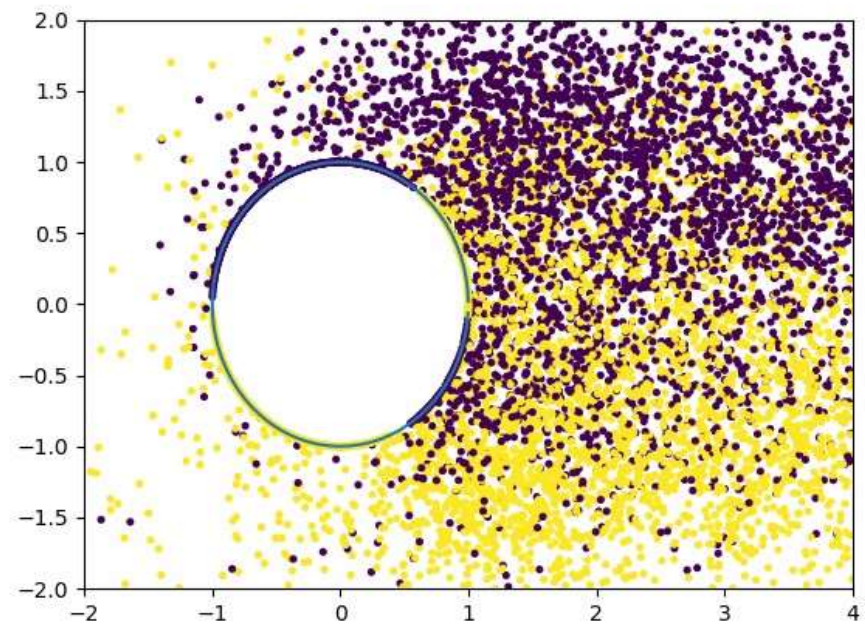
- b) At 1 sec



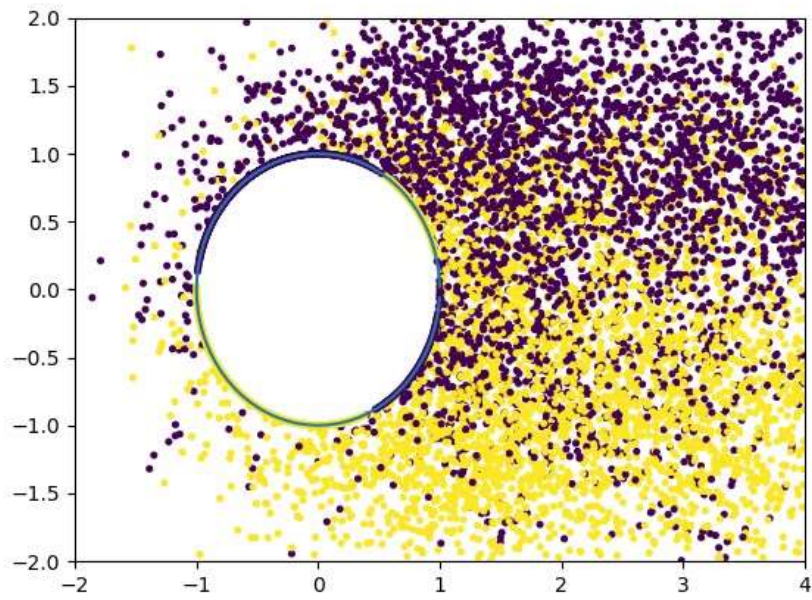
c) At 2 sec



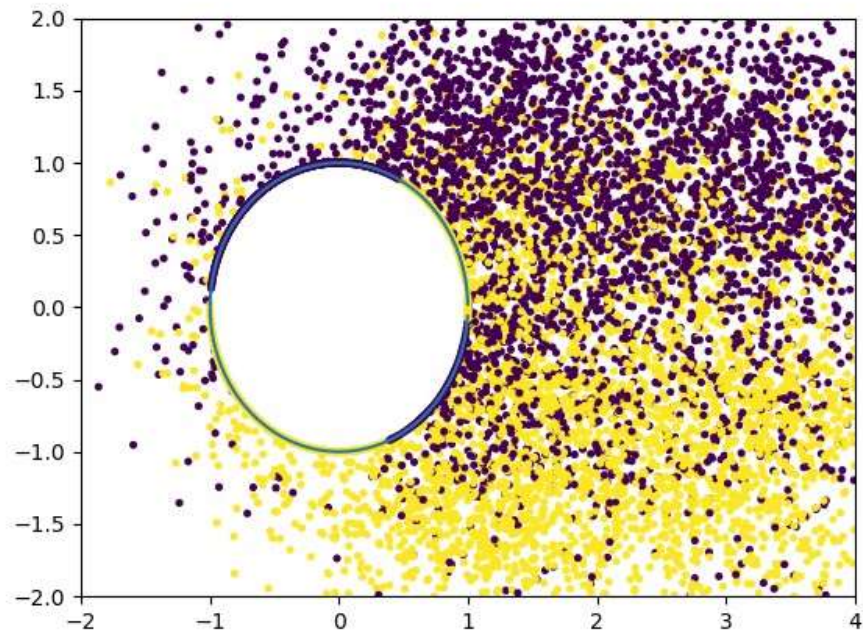
d) At 3 sec



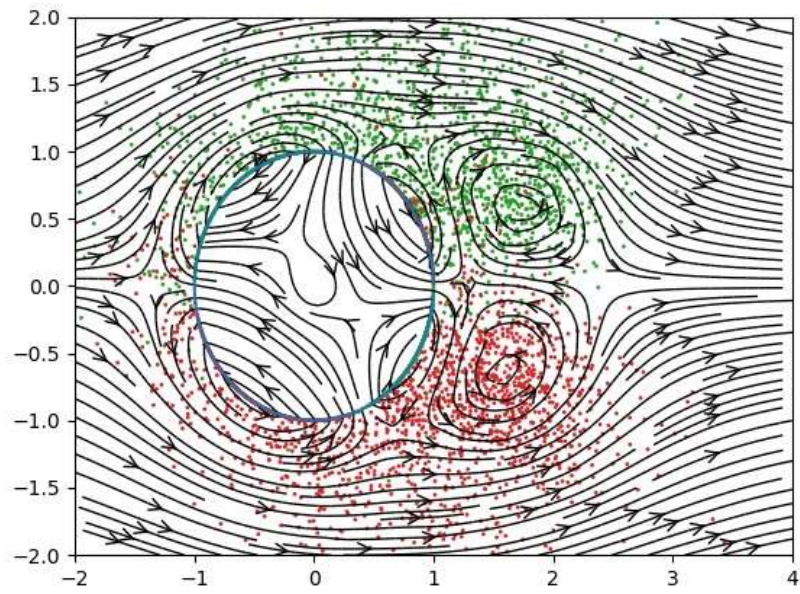
e) At 4 sec



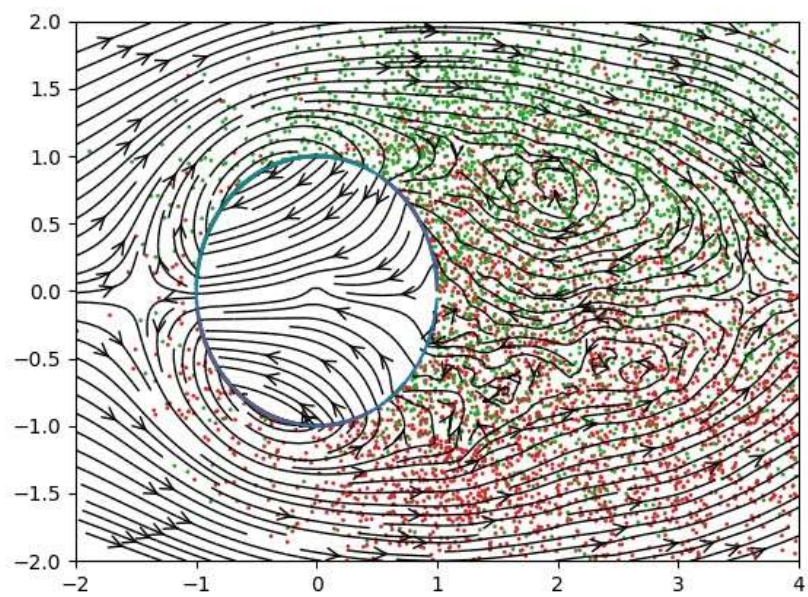
f) At 5 sec



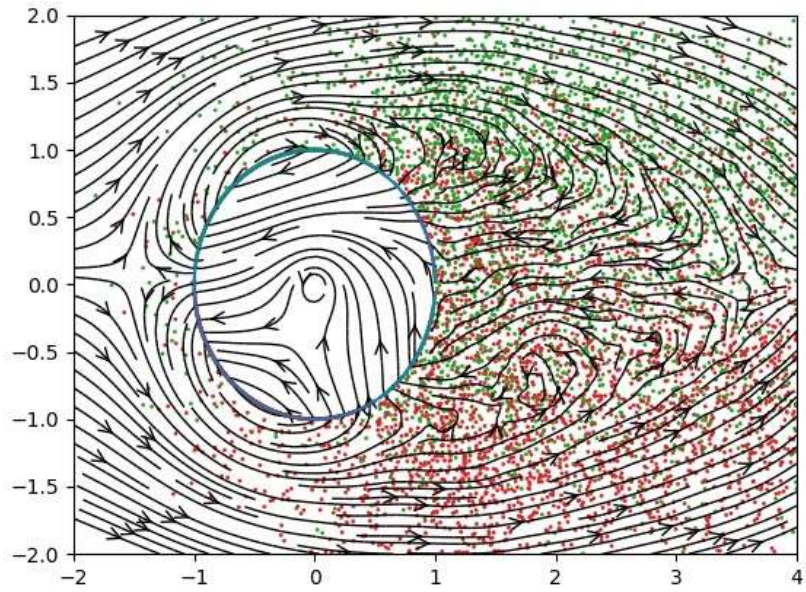
2) Velocity field
a) Just after start



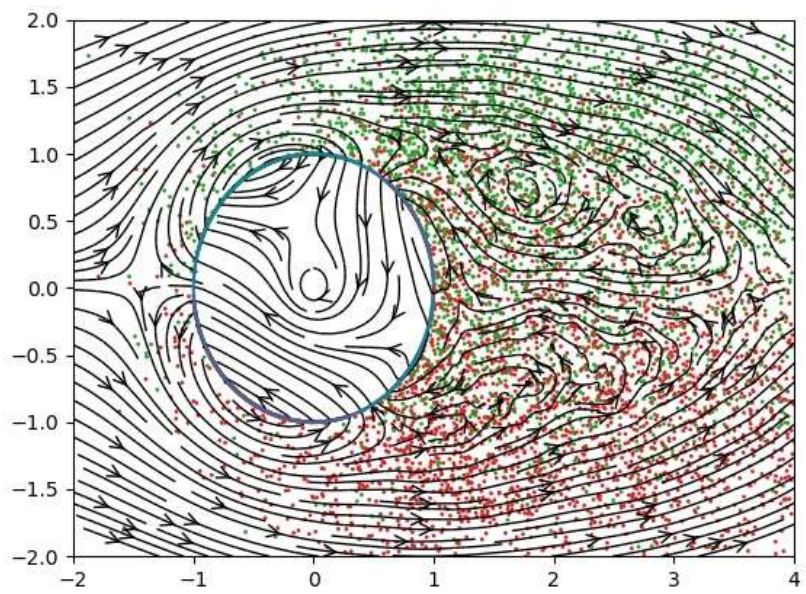
b) 1 sec



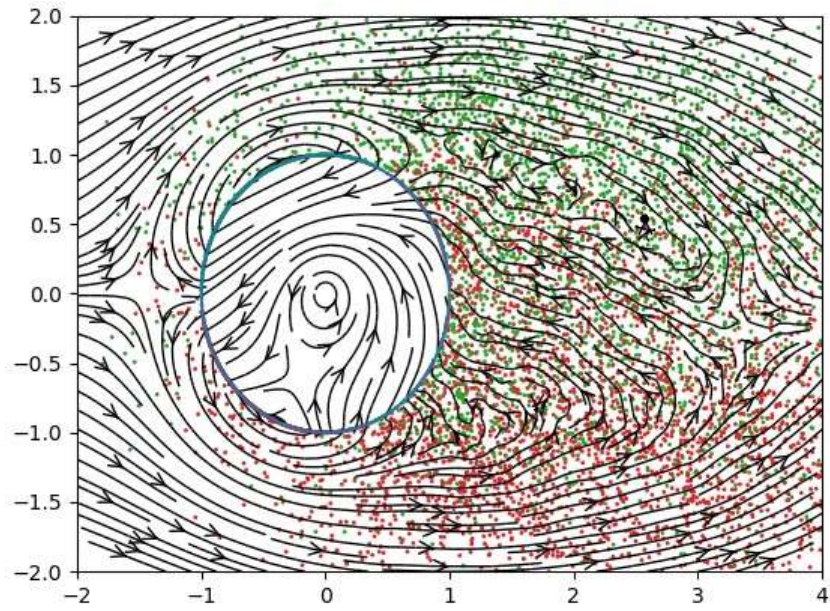
c) 2 sec



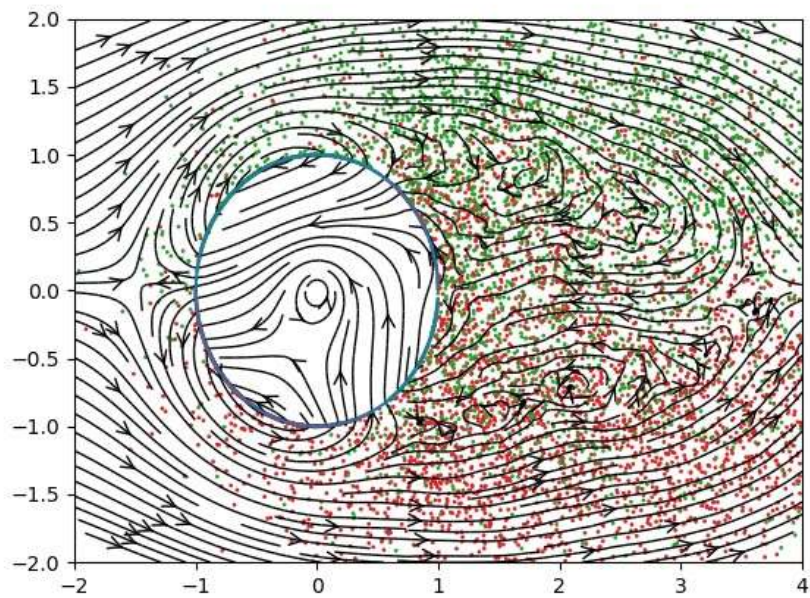
d) 3 sec



e) 4 sec



f) 5 sec



```

1  import numpy as np
2  import matplotlib.pyplot as plt
3
4  TOS=5
5  no_f_p=200
6  no_pan=75
7  delta_sq=(2.0/no_pan)**2
8
9  velocit_y=1.0
10 Re=1000.0
11 vis=2.0*velocit_y/1000
12 Gmax=0.1
13
14
15 def free_stream(free_vel,nop):
16     return velocit_y*free_vel*np.ones(nop)
17
18
19 def Sphere_body(no_pan,center,radius,b_vel):
20     theta=np.linspace(0.0,2*np.pi*(1-1/no_pan),no_pan)
21     p_co = center + radius*np.exp(1j*theta)
22     p_mid= center + (radius*np.cos(np.pi/no_pan))*np.exp(1j*(theta+(2/4)*2*np.pi/no_pan))
23     u_norm= p_mid/np.abs(p_mid)
24     p_vel= b_vel*np.ones(no_pan)
25     p_len= 2*radius*np.sin(np.pi/no_pan)*np.ones(no_pan)
26     p_action = p_mid + (p_len/4.0)*u_norm*np.exp(1j*np.pi/2)
27     p_control = p_mid + (p_len/4.0)*u_norm*np.exp(-1j*np.pi/2)
28     return p_action,p_control,u_norm,p_vel,p_len,p_co
29
30 #####
31
32 def vel_panel_panel(i,z_i,j,p_action,u_norm,p_len,no_pan):
33     l=p_len[j]
34     theta=np.angle(u_norm[j])-np.pi/2
35     z=(z_i-p_action[j])*np.exp(-1j*theta)
36     vel=0.0+0.0j*np.zeros(no_pan)
37     index=np.arange(no_pan)
38     vel[index] = (1.0 + ( z/p_len + 1/4.0 ) * np.log((z-3*l/4)/(z+l/4)))
39     vel[(index+1)%no_pan] += (-1.0 + ( -z/p_len + 3.0/4.0 ) * np.log((z-3*l/4)/(z+l/4)))
40
41     vel=np.complex(0,0.5/(np.pi))*vel
42     vel=np.conjugate(vel)*np.exp(1j*theta)
43     return vel
44
45
46 def A_cal(p_action,p_control,u_norm,p_len,no_pan):
47     A=np.array([0.0j*np.zeros(no_pan) for j in range(no_pan)])
48     for i in range(no_pan):
49         j=np.arange(no_pan)
50         z_i=p_control[i]
51         A[i]=vel_panel_panel(i,z_i,j,p_action,u_norm,p_len,no_pan)
52     return A
53
54
55 def tau_matrix(A,u_norm,no_pan):
56     for i in range(no_pan):
57         A[i]=(A[i].real * u_norm[i].real) + (A[i].imag * u_norm[i].imag)
58     return A
59
60
61 def tau_cal(A,v_w,u_norm,p_vel):
62     v_b_w = p_vel - v_w
63     b = (v_b_w.real * u_norm.real) + (v_b_w.imag * u_norm.imag)
64     Gam=np.dot(np.linalg.inv(A),b)
65     return Gam
66
67

```

```

68 def panel_method(v_w,p_action,p_control,u_norm,p_len,no_pan,p_vel):
69     A=A_cal(p_action,p_control,u_norm,p_len,no_pan)
70     A=tau_matrix(A,u_norm,no_pan)
71     Gam=tau_cal(A,v_w,u_norm,p_vel)
72     return Gam.real
73
74
75 def vel_due_panel(z_i,p_action,u_norm,p_len,no_pan,Gam):
76     theta=np.angle(u_norm)-np.pi/2
77     z=(z_i-p_action)*np.exp(-1j*theta)
78     l=p_len
79     index=np.array(np.arange(no_pan))
80     vel = Gam[index]*(1.0 + ( z/p_len + 1/4.0 ) * np.log((z-3.0*l/4.0)/(z+l/4.0)))
81     vel+= Gam[(index+1)%no_pan]*(-1.0 + ( -z/p_len + 3.0/4.0 ) * np.log((z-3*l/4)/(z+l/4)))
82     vel=np.complex(0,0.5/(np.pi))*vel
83     vel=np.sum(np.conjugate(vel)*np.exp(1j*theta))
84     return vel
85
86
87 def tot_circ_zero(z_i,Gam,p_len,no_pan):
88     if z_i==0.0+0.0j:
89         return 0.0
90     else:
91         tau_center=-np.sum(Gam*p_len)
92         vel=np.complex(0,-0.5/(np.pi))*tau_center/(z_i)
93         return vel.conjugate()
94
95
96 #####
97 # No slip
98 #####
99
100 def ini_slip_vel(z_i,p_action,u_norm,p_len,no_pan,Gam,p_vel):
101     v_slip = vel_due_panel(z_i,p_action,u_norm,p_len,no_pan,Gam)+tot_circ_zero(z_i,Gam,
102     p_len,no_pan)
103     v_slip += free_stream(1.0,1)
104     v_slip += -p_vel
105     return v_slip
106
107 def slip_vel(z_i,p_action,u_norm,p_len,no_pan,Gam,Pos,tau,delta_sq,p_vel):
108     v_slip = vel_due_panel(z_i,p_action,u_norm,p_len,no_pan,Gam)+tot_circ_zero(z_i,Gam,
109     p_len,no_pan)
110     v_slip += free_stream(1.0,1) + vel_due_vortex(z_i,Pos,tau,delta_sq)
111     v_slip += -p_vel
112     return v_slip
113
114 def No_of_blob(v_slip,p_action,u_norm,p_len):
115     tangent = u_norm*np.exp(1j*np.pi/2)
116     v_dot_t = v_slip.real * tangent.real + v_slip.imag * tangent.imag
117     no_of_blob = np.abs(2*v_dot_t*p_len/Gmax)
118     return np.int(no_of_blob),np.sign(v_dot_t)
119
120 def Blobs_config(no_of_blob,Sign,u_norm,p_len,p_co):
121     if no_of_blob==0:
122         return
123     else:
124         d_of_blob = p_len/no_of_blob
125         start_pos = p_co + 0.0*u_norm + 0.0*0.5*d_of_blob*u_norm + ( np.arange(no_of_blob)
126         ) * d_of_blob + 0.5*d_of_blob *(u_norm*np.exp(1j*np.pi/2))
127         return start_pos,np.ones(no_of_blob)*Sign[0]*Gmax
128
129 def ini_all_blobs(p_action,u_norm,p_len,no_pan,Gam,p_co,p_vel):
130     new_POS=np.empty(shape=0)
131     new_TAU=np.empty(shape=0)
132     for i in range(no_pan):
133         z_i=p_action[i]

```

```

132     v_slip = ini_slip_vel(z_i,p_action,u_norm,p_len,no_pan,Gam,p_vel[i])
133     no_of_blob,Sign = No_of_blob(v_slip,p_action[i],u_norm[i],p_len[i])
134     if Blobs_config(no_of_blob,Sign,u_norm[i],p_len[i],p_co[i])==None:
135         pass
136     else:
137         new_pos,new_tau = Blobs_config(no_of_blob,Sign,u_norm[i],p_len[i],p_co[i])
138         new_POS=np.append(new_POS,new_pos)
139         new_TAU=np.append(new_TAU,new_tau)
140     return new_POS,new_TAU
141
142 def all_blobs(p_action,u_norm,p_len,no_pan,Gam,Pos,tau,delta_sq,p_co,p_vel):
143     new_POS=np.empty(shape=0)
144     new_TAU=np.empty(shape=0)
145     for i in range(no_pan):
146         z_i=p_action[i]
147         v_slip = slip_vel(z_i,p_action,u_norm,p_len,no_pan,Gam,Pos,tau,delta_sq,p_vel[i])
148         no_of_blob,Sign = No_of_blob(v_slip,p_action[i],u_norm[i],p_len[i])
149         if Blobs_config(no_of_blob,Sign,u_norm[i],p_len[i],p_co[i])==None:
150             pass
151         else:
152             new_pos,new_tau = Blobs_config(no_of_blob,Sign,u_norm[i],p_len[i],p_co[i])
153             new_POS=np.append(new_POS,new_pos)
154             new_TAU=np.append(new_TAU,new_tau)
155     return new_POS,new_TAU
156
157
158 def vortex_vel(Pos,tau,delta_sq,p_action,u_norm,p_len,no_pan,Gam):
159     N=(np.shape(Pos))[0]
160     vel = np.zeros_like(Pos)
161     for i in range(N):
162         z_i=Pos[i]
163         vel[i] = vel_due_vortex(z_i,Pos,tau,delta_sq) +free_stream(1.0,1)
164         vel[i] += vel_due_panel(z_i,p_action,u_norm,p_len,no_pan,Gam)+tot_circ_zero(z_i,
165             Gam,p_len,no_pan)
166     return vel
167
168 def vel_due_vortex(z_i,Pos,tau,delta_sq):
169     r_d=z_i-Pos
170     check_1=np.where(np.abs(r_d)>=delta_sq)
171     check_2=np.where(( np.abs(r_d)<delta_sq) & (np.abs(r_d)!=0.0))
172     check_3=np.where(np.abs(r_d)==0.0)
173     vel=np.asarray(r_d)
174     vel[check_1]=np.complex(0,0.5/(np.pi))*tau[check_1]*r_d[check_1]/((np.abs(r_d[check_1]
175         ]))*2)
176     vel[check_2]=np.complex(0,0.5/(np.pi))*tau[check_2]*r_d[check_2]/(np.abs(r_d[check_2
177         ]*delta_sq))
178     vel[check_3]=0.0
179     return np.sum(vel)
180
181 def vortex_w(no_pan,p_control,Pos,tau,delta_sq):
182     v_w=np.zeros(no_pan)*(0.0+0.0j)
183     for i in range(no_pan):
184         z_i=p_control[i]
185         v_w[i]=vel_due_vortex(z_i,Pos,tau,delta_sq)
186     v_w += free_stream(1.0,no_pan)
187     return v_w
188
189 def random_walk(pos,d_t):
190     sigma=np.sqrt(2*vis*d_t)
191     pos=np.random.normal(0.0,np.sqrt(sigma),2*np.size(pos)).view(np.complex)+pos
192     return pos*(1.0+0j)
193
194 def reflection(pos):
195     index=np.where(np.abs(pos)<1.0)

```

```

196     dist=2.0-np.abs(pos[index])
197     pos[index]=dist*(pos[index])/np.abs(pos[index])
198     return pos
199
200 def par_plot(pos,tau):
201     pos_ind=np.where(tau>=0)
202     neg_ind=np.where(tau<0)
203     dummy_tau=np.asarray(tau,dtype=np.str)
204     dummy_tau[pos_ind]= 'tab:red'
205     dummy_tau[neg_ind]= 'tab:green'
206     return dummy_tau
207
208 def recons(pos,tau):
209     index_1=np.where( (np.abs(pos.real)<4) & (np.abs(pos.imag)<2))
210     pos=pos[index_1]
211     tau=tau[index_1]
212     return pos,tau
213
214 def velocity_feild(Pos,tau,Gam,delta_sq,p_len,no_pan,p_action,u_norm):
215     x = np.linspace(-2, 4, 300)
216     y = np.linspace(-2, 2, 100)
217     X, Y = np.meshgrid(x, y)
218     z = X+1j*Y
219     vel_feild=np.asarray(z)
220     for i in range(np.shape(z)[0]):
221         for j in range(np.shape(z)[1]):
222             z_i=z[i,j]
223             vel_feild[i,j] = vel_due_vortex(z_i,Pos,tau,delta_sq) + free_stream(1.0,1)
224             vel_feild[i,j] += tot_circ_zero(z_i,Gam,p_len,no_pan) + vel_due_panel(z_i,
                p_action,u_norm,p_len,no_pan,Gam)
225     return x,y,vel_feild
226
227 def lift_drag(pos,tau):
228     return np.sum( tau * ( pos.imag - 1j* pos.real) )
229
230
231 def Vortex_simulation(d_t,no_pan):
232
233     p_action,p_control,u_norm,p_vel,p_len,p_co = Sphere_body(no_pan,0+0j,1.0,0.0+0.0j)
234
235     Gam=panel_method(free_stream(1.0,no_pan),p_action,p_control,u_norm,p_len,no_pan,p_vel
        )
236     Pos,tau=ini_all_blobs(p_action,u_norm,p_len,no_pan,Gam,p_co,p_vel)
237     tim=0.0
238
239     I = np.empty(shape=0)
240
241     while tim<TOS:
242
243
244         Pos = random_walk(Pos,d_t/2.0)
245         Pos = reflection(Pos)
246
247         print(tim,np.size(Pos))
248
249         v_w=vortex_w(no_pan,p_control,Pos,tau,delta_sq)
250         Gam=panel_method(v_w,p_action,p_control,u_norm,p_len,no_pan,p_vel)
251         Pos_temp = Pos + 0.5*(vortex_vel(Pos,tau,delta_sq,p_action,u_norm,p_len,no_pan,
            Gam))*d_t
252
253         v_w=vortex_w(no_pan,p_control,Pos_temp,tau,delta_sq)
254         Gam=panel_method(v_w,p_action,p_control,u_norm,p_len,no_pan,p_vel)
255         Pos += (vortex_vel(Pos_temp,tau,delta_sq,p_action,u_norm,p_len,no_pan,Gam))*d_t
256
257         Pos = random_walk(Pos,d_t/2.0)
258         Pos = reflection(Pos)
259

```

```

260     v_w=vortex_w(no_pan,p_control,Pos,tau,delta_sq)
261     Gam=panel_method(v_w,p_action,p_control,u_norm,p_len,no_pan,p_vel)
262     Pos_ins,tau_ins=all_blobs(p_action,u_norm,p_len,no_pan,Gam,Pos,tau,delta_sq,p_co,
    p_vel)

263
264     Pos=np.append(Pos,Pos_ins)
265     tau=np.append(tau,tau_ins)
266
267     Pos,tau=recons(Pos,tau)
268
269     I=np.append(I,lift_drag(Pos,tau))
270
271     """
272     plt.figure(1)
273     plt.plot(p_control.real,p_control.imag)
274     color_par = par_plot(Pos.copy(),tau.copy())
275     plt.scatter(Pos.real,Pos.imag,s=1,c=color_par)
276     x,y,vel_feild =
    velocity_feild(Pos.copy(),tau.copy(),Gam.copy(),delta_sq,p_len.copy(),no_pan,p_ac
    tion.copy(),u_norm.copy())
277     plt.streamplot(x, y, vel_feild.real,vel_feild.imag ,color='k', linewidth=1,
    cmap=plt.cm.inferno,density=2, arrowstyle='->', arrowsize=1.5)
278     plt.xlim(-2.0,4.0)
279     plt.ylim(-2.0,2.0)
280     plt.savefig("velocit_y"+str(np.round(tim,1))+".png")
281     plt.close(1)
282
283     """
284
285     p_action += p_vel*d_t
286     p_control+= p_vel*d_t
287
288     tim += d_t
289
290     return Pos,tau,Gam,I,p_action,p_control
291
292
293
294 def path_plotting(d_t):
295     plt.figure(1)
296     p_action,p_control,u_norm,p_vel,p_len,p_co=Sphere_body(no_pan,0+0j,1.0,0.0)
297     Pos,tau,Gam,I,p_action,p_control=Vortex_simulation(d_t,no_pan)
298     plt.plot(p_control.real,p_control.imag)
299     color_par = par_plot(Pos.copy(),tau.copy())
300     plt.scatter(Pos.real,Pos.imag,s=1,c=color_par)
301     x,y,vel_feild = velocity_feild(Pos.copy(),tau.copy(),Gam.copy(),delta_sq,p_len.copy
    (),no_pan,p_action.copy(),u_norm.copy())
302     plt.streamplot(x, y, vel_feild.real,vel_feild.imag ,color='k', linewidth=1, cmap=plt.
    cm.inferno,density=2, arrowstyle='->', arrowsize=1.5)
303     plt.xlim(-2.0,4.0)
304     plt.ylim(-2.0,2.0)
305     plt.show()
306
307
308 #Vortex_simulation(0.1,75)
309
310 def C_D(d_t):
311     Pos,tau,Gam,I,p_action,p_control=Vortex_simulation(d_t,no_pan)
312     c_d = 2*I/(velocit_y)**2
313     c_d_temp_1=c_d.copy()
314     c_d_temp_2=c_d.copy()
315     for i in range(np.shape(c_d)[0]-4):
316         c_d_temp_1[i] = ( c_d[i] + c_d[i+1] + c_d[i+2] + c_d[i+3] ) * 0.25
317         j=np.shape(c_d)[0] - i - 1
318         c_d_temp_2[j] = ( c_d[j] + c_d[j-1] + c_d[j-2] + c_d[j-3] ) * 0.25
319     c_d = (c_d_temp_1 + c_d_temp_2)* 0.5
320     Force = (c_d[1:]-c_d[:-1])/d_t

```

```
321     plt.figure(1)
322     plt.plot(Force.real, 'r', label='C_d')
323     plt.legend(loc='best')
324     plt.show()
325
326     #path_plotting(0.1)
327
```